

注册表、编译器与云内存：迈向以 Agent 操作系统为核心的新一代智能体架构

核心的新一代智能体架构

技术白皮书 · V3.1 最终发布版

作者：GCat/0101

发布日期：2026 年 6 月 18 日

领域：智能体架构、分布式系统、操作系统设计

核心定位：Agent 互联网基础设施规范

摘要

当前基于大语言模型的自主智能体，正从对话式交互向复杂任务执行快速演进，但其记忆与通信机制仍深陷“上下文即记忆”的原始范式陷阱：系统指令、角色设定、历史对话、检索片段与工具输出无差别混合注入线性上下文窗口，导致状态管理模糊、计算资源冗余、上下文污染频发，并引发数据隐私与商业可持续性的双重困境。业界长期在应用层通过提示词工程、检索增强生成、记忆摘要与上下文扩容等手段进行局部优化，始终未能突破“全量数据搬运”的底层逻辑。

本文跳出应用层补丁思维，将经典操作系统的分层抽象、结构化状态管理、编译式通信与资源隔离思想系统性迁移至智能体设计空间，提出完整的 **AgentOS 架构**。该架构构建了“本地记忆层 - 编译通信层 - 云端持久层”的三级分层体系，核心创新包含三大支柱：(1) **四表分离式本地记忆系统**：将 Agent 记忆拆解为注册表、环境变量、系统日志、记忆库四张逻辑独立的表，分别承载确定性元数据、运行时参数、行为审计记录与原始非结构化数据，实现状态的常数级确定性查询与数据物理隔离。(2) **编译化工单通信协议**：由独立编译器担任意图净化与指令翻译角色，将人机自然语言交互与机机结构化指令通信彻底分离，通过标准化工单实现最小必要信息传输，从架构层面解决上下文污染与重复传输问题。(3) **云内存指针服务**：颠覆“云存储 - 本地搬运 - 再上传”的低效链路，将认知资产直接驻留在推理引擎同址的专用内存空间，通过显式缓存握手协议实现可编程的持久化上下文，达成认知资产与计算资源的同址协同。

本文严格定义了 AgentOS 的操作系统原语、进程管理模型、中断处理机制、分布式

CAP 权衡与完整威胁模型，补全了从系统调用接口、注册表 Schema 定义、工单协议完整规范到编译器参考实现的全部工程细节，并设计了包含消融实验的标准化评估框架以验证各组件的独立贡献。AgentOS 不仅将记忆访问效率与状态确定性提升至数量级，更以“配置在云、材料在端、指针隔离”的架构级隐私哲学，开辟了从“按 Token 计费”到“按智能密度订阅”的全新商业范式。本文论证，AgentOS 并非单一技术优化，而是为构建 Agent 互联网奠定基础设施的范式革命，具备成为下一代智能体行业标准的潜力。

关键词：智能体架构；Agent 操作系统；状态管理；上下文优化；分层记忆；编译通信；云内存；隐私保护；多 Agent 协作

1. 引言

1.1 “仓库与货车”：一个必须打破的悖论

想象一个仓库管理员。他的仓库里有数以百万计的货物，分类清晰、摆放整齐。但每当接到客户订单——哪怕只订购一件商品——他都必须驾驶一辆巨型货车，把仓库里**所有相关货物**全部装车，送到客户面前，让客户自行翻找所需商品。下一次新订单，依然重复全量装载的流程。

这，就是当前绝大多数 AI Agent 记忆系统的真实写照。

“仓库”是精心设计的记忆系统：非结构化长文本摘要、向量检索库、知识图谱。“货车”是大模型的上下文窗口。“订单”是用户的每一次请求。业界倾注巨大心力优化仓库：更精准的语义检索、更合理的记忆衰减曲线、更精细的实体关系抽取。这些努力本身极具价值，但它们共同陷入了一个盲区——**从未质疑“必须把所有货物装车”这一行为本身**。

记忆越精细，需要注入上下文的“货物”就越多，上下文越臃肿，算力消耗越大，成本越高。大模型厂商的回应是提供越来越大的上下文窗口——从 4K 到 128K 再到 1M Token——这无异于给管理员换更大的货车，却未改变全量搬运的本质。记忆无限增长，窗口终有上限，且上下文越长，推理成本越高、延迟越大，模型在长上下文中对关键信息的召回准确率反而下降。这是一条饮鸩止渴的路径，我们将其称为智能体的“记忆墙”困境。

1.2 系统层思维的缺失

这一困境的根源，在于将 Agent 仅仅视为一个“应用”。我们在应用层用提示词工程、检索增强生成、记忆摘要等技巧模拟人类记忆表象，却从未从“操作系统”的视角审视 Agent 的软件栈。

回顾个人计算的演进史。在 DOS 时代，用户每次输入命令，都必须在脑海中完整重建计算机的当前状态：当前目录、可用文件、上次操作结果。系统没有统一的状态管理，

所有状态都依赖用户记忆。这和今天 Agent 通过阅读冗长上下文来“回忆”自己的状态何其相似！现代操作系统的革命性突破，在于提供了**抽象与隔离**：文件系统管理持久化（用户无需知道文件的物理扇区），注册表存储系统和应用的配置元数据（修改配置无需改动代码），进程管理器维护运行时状态和资源分配（每个进程拥有独立受保护的内存空间），系统日志独立记录行为审计。这些组件职责分明，通过高效的系统调用通信，而非互相阅读对方的“长篇文本”。当你在 Windows 11 上点击一个图标，系统不是去读一本十万字的《系统使用说明书》，而是仅查询注册表中一个轻量的键值，即可完成全部状态加载。

Agent 缺少的，正是这一层“系统”。

1.3 本文的核心命题与贡献

本文提出核心命题：**应为自主智能体构建一套与经典操作系统同构的记忆与通信基础设施，即 AgentOS**。这并非粗糙的类比，而是一套可被严格定义、可工程化落地、可验证评估的完整架构原则。

本文的核心贡献如下：

- 本地记忆的分层重构**：提出“注册表 - 环境变量 - 系统日志 - 记忆库”四表分离模型，将确定性元数据、运行时参数、行为审计记录与原始非结构化数据彻底解耦，实现状态的常数级确定性读取。其中，注册表是核心创新——它不同于知识图谱（试图描述万物关系），而是只记录“我”和“我的环境”的配置与状态，通过树形层级键值对实现原子性读写。
- 通信的编译范式转移**：提出“编译器 + 工单协议”双层通信架构，将自然语言交互与结构化机间通信分离，实现意图净化、指令标准化与最小必要信息传输，从架构层面解决上下文污染与重复传输。编译器采用 L1 规则 + L2 模型的两级实现，高频简单操作零模型调用成本。
- 云端的内存革命**：提出云内存指针服务，颠覆“云存储 - 本地搬运 - 再上传”的低效链路，实现认知资产与推理引擎同址驻留，通过显式缓存握手协议实现可编程的持久化上下文，定义了 L1 软件定义→L2 近推理内存池→L3 同址融合的三层递进落地路径。
- 架构级隐私安全模型**：通过“配置与材料分离、指针与数据分离、陪伴与执行分离”三大原则，在不依赖复杂加密算法的前提下，于架构层面实现原生隐私保护——云端只存指针和配置，原始数据始终在本地。
- 完整的系统抽象与评估体系**：补全进程管理、系统调用、中断处理等操作系统核心原语，明确分布式 CAP 权衡与威胁模型，并设计包含消融实验的标准化评估框架。
- 全新的商业与生态范式**：论证 AgentOS 如何推动行业从“按 Token 计费”转向“按智能密度订阅”，构建以云内存认知资产为核心的强锁定商业护城河，并为多 Agent 协

作互联网奠定工单协议基础。

2. 相关工作

Agent 记忆与通信架构的研究，横跨自然语言处理、多智能体系统、操作系统与分布式系统多个领域。本章按四条技术路线梳理现有方案，并精准定位 AgentOS 的创新边界。

2.1 长上下文与虚拟内存方案

以 MemGPT [2] 为代表的“LLM as OS”思路，通过主上下文与外部存储的分页调度，模拟操作系统的虚拟内存机制，突破了固定上下文长度的限制。其核心机制是将对话历史分为“主上下文”和“外部存储”两层，当信息超出主上下文容量时，通过模型自主调用函数将过期内容换出到外部存储，需要时再检索换入。然而，这类方案本质仍是**单一一大上下文的时分复用**：所有信息（系统指令、记忆摘要、检索片段）依然以自然语言形式混合在同一上下文内，状态切换完全依赖模型自身对摘要文本的语义解读，而非确定性的结构化查询。其核心优化点是上下文长度的扩展，而非记忆架构的重构。此外，模型自主调度记忆换入换出本身也消耗 Token 和推理资源，在长任务中调度开销不可忽视。

AgentOS 在此基础上迈出关键一步：将确定性状态信息从上下文中完全剥离，构建独立的注册表层，通过编译器实现结构化的读写操作，彻底消除语义解读的不确定性，实现了从“虚拟内存模拟”到“完整操作系统架构”的跃迁。AgentOS 的注册表查询是常数级的键值寻址，不消耗模型推理资源。

2.2 结构化记忆方案

知识图谱记忆（如 GraphRAG [24]）与实体记忆尝试将对话内容抽取为三元组（主体，关系，客体）或结构化属性对，单独存储以提升查询效率与推理能力。GraphRAG 通过将文档预处理为实体 - 关系网络，在推理时进行图遍历检索。这类方案试图对**所有信息强制结构化**，导致抽取成本随对话规模指数上升——开放域场景下命名实体识别和关系抽取本身错误率就不低，错误还会在后续抽取中累积传播。同时，三元组结构难以捕捉自然语言中的模糊性、反讽与隐含语义，陷入“过度工程化”的困境。

AgentOS 采取截然不同的策略：**仅对确定性元数据（配置、状态、指针）进行结构化，原始非结构化数据保留在记忆库中按需调用**。注册表不是微型知识图谱，它不描述“用户与项目 X 的关系”，而是记录

`HKEY_CURRENT_USER\ProjectRegistry\Proj_X\Status = "Active"`这样的自身配置。这既获得了结构化查询的效率（常数级键值寻址），又避免了全面结构化的工程灾难。

2.3 多 Agent 通信协议

传统多智能体系统的 FIPA-ACL 协议 [23] 定义了复杂的语义消息格式，包含发送者、接收者、内容语言、本体等字段，支持请求、告知、承诺等多种言语行为。但实际落地困难，因其依赖接收方对消息进行复杂的语义解析，且协议本身过于笨重——每条消息的元数据开销就可能超过实际内容。近年来大模型 Agent 之间的协作多采用自然语言对话通信，虽然灵活性好，但信息密度低、歧义性高，通信效率低下。

AgentOS 的工单协议走出了第三条路径：基于编译器前置净化机制，用户的自然语言先经编译器转化为结构化指令，再发送给执行模型或其他 Agent。这保留了自然语言作为人机入口的灵活性，但在机机通信层面用结构化协议替代，大幅降低通信开销与歧义。协议的 `instruction.op` 字段直接声明操作类型，接收方无需“理解”意图即可执行。

2.4 上下文缓存技术

2024 年以来，谷歌、Anthropic 等厂商相继推出提示缓存功能 [10,12]。谷歌 Vertex AI 的 Context Caching 允许用户指定需缓存的内容，服务端在 TTL 内复用其 KV 张量；Anthropic 的 Prompt Caching 自动识别重复前缀并缓存。这一技术方向证实了我们对“重复计算浪费”的洞察，但现有方案存在本质局限：缓存策略（淘汰时机、存活时间、生效范围）由厂商黑盒控制，开发者无法将其作为可信赖的持久化记忆层——你不知道缓存什么时候会被驱逐；缓存内容为原始提示词块，而非结构化的、可直接注入的认知资产；各家 API 互不兼容，无法形成跨平台标准。

AgentOS 将这种能力提升为显式的、可编程的云内存协议。开发者通过标准化指令 `cache_store/cache_load` 自主管理认知资产的全生命周期（创建、引用、更新、失效），服务端对缓存的生命周期给出明确承诺（TTL 保障），缓存内容为编译后的结构化认知资产，实现了从“被动优化”到“主动架构”的升级。

2.5 现有方案横向对比

方案类型	代表产品	核心范式	状态确定性	上下文效率	隐私保护	生态开放性
纯长上下文	GPT-4 128K	上下文即记忆	极低，完全依赖语义解读	极低，全量重传	差，原始数据全上传	高，通用接口
应用层框架	LangChain	上下文 + RAG 增	低，依赖记忆摘要	低，检索片段注入	差，需上传切片	高，开源通用

		强				
虚拟内存方案	MemGPT	上下文分页调度	中，依赖模型调度	中，分页换入换出	中，数据本地存储	中，特定架构
托管式 Agent	Assistants API	服务端上下文管理	中（黑盒）	中，服务端复用	差，数据留存在厂商	低，厂商锁定
结构化记忆	GraphRAG	三元组知识图谱	较高，图查询	中，结构化结果注入	中，需上传抽取	中，特定场景
AgentOS	-	操作系统分层架构	极高，结构化键值查询	极高，仅传工单与增量	极高，架构级隔离	高，开放协议标准

3. AgentOS 核心设计哲学

3.1 架构优先于算法

这是 AgentOS 的第一性原则。现有方案倾向于用复杂的算法去修正一个根本上有缺陷的架构——用更精巧的检索排序、更复杂的记忆权重调整去弥补“全量上下文注入”的结构性低效。然而架构决定了系统组件的职责与边界：一个错误的架构，内部算法再精妙，也难以突破本质瓶颈；一个正确的架构，却可以用极其简单的算法实现卓越的性能。经典操作系统的分页虚拟内存机制，基于简单的页表映射与 LRU 置换策略，其效率远胜于任何在实模式下设计的复杂内存分配算法。

AgentOS 的设计重心，始终放在定义组件及其边界关系上，而非具体的记忆评分公式。用最简单的结构化键值查询（常数级复杂度），就能实现比最复杂的语义检索更高的状态确定性——这正是架构的力量。

3.2 分层优于混合

“混合”是当前架构的万恶之源。系统指令、角色设定、历史对话、检索片段、工具输出全部作为无差别的 Token 序列混合在同一个上下文窗口中，造成职责不清、污染严重、难以调试。当 Agent 行为异常时，你无法快速定位是系统提示词被污染、检索到了错误片段、还是历史对话中的某段闲聊干扰了判断。AgentOS 严格遵循分层原则，

将 Agent 的完整能力栈拆解为本地记忆层、编译通信层与云端持久层。每一层内部又进一步细分组件，组件之间通过定义清晰的系统调用接口交互。这种分离使得每个组件可以独立演化、独立优化，并在出现问题时快速定位根源。

3.3 指针优于数据

这是贯穿 AgentOS 所有数据设计的核心思想。我们不移动数据，而是移动对数据的引用。在传统 Agent 架构中，获知“用户偏好简洁风格”需要将完整描述文本注入上下文；在 AgentOS 中，这段描述存储在记忆库中，运行时只传递一个指针

`user_pref_style_id = 0x4A`。注册表键值、环境变量、云内存缓存 ID，本质都是不同层级的指针。它们极小（通常为字符串或数字 ID）、极快，并且天然支持版本控制、访问控制、失效处理等管理操作。更重要的是，指针天然实现了引用与数据的物理隔离——即使指针泄露，攻击者也无法通过指针获取原始数据。

3.4 协议优于提示

提示词是给人看的，模糊且依赖语境。当交互双方从“人与模型”变为“系统与模型”时，再用提示词就是低效且不可靠的——模型可能“误解”提示词的意图，提示词之间可能产生冲突，且提示词的编写本身就是一门“玄学”。AgentOS 主张在系统内部通信中使用结构化的协议。一个编译器可以将用户模糊的指令“帮我继续上次的分析”转化为精确的协议指令`{op: "RESUME_TASK", task_id: 0x334, params: {...}}`，模型接收后无需“理解”什么是“继续”，只需按照协议规范执行对应操作即可。这与 CPU 执行机器码而非阅读英文说明书，是同一个道理——机器之间的通信，永远是结构化协议效率更高、确定性更强。

4. 本地记忆系统设计：四表分离与 OS 抽象

本地记忆层是 AgentOS 的用户态运行空间，也是整个系统的状态真值源。本章详细阐述四表分离记忆模型、操作系统原语与系统调用接口、进程管理模型以及中断处理机制。

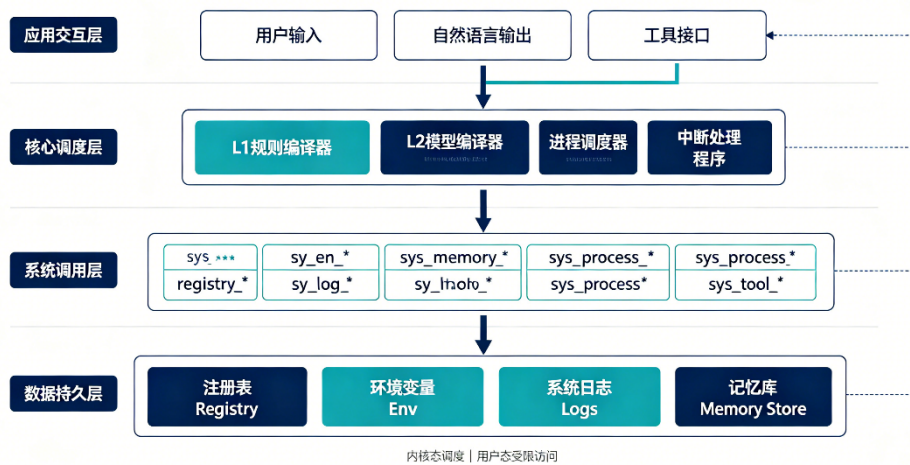
4.1 系统总体架构

AgentOS 本地环境采用分层设计，从下至上依次为：

1. **数据持久层**：四表分离的存储系统，包括注册表、环境变量、系统日志、记忆库；
2. **系统调用层**：标准化的内核调用接口，实现对底层资源的受控访问；
3. **核心调度层**：编译器担任调度器角色，负责进程管理、中断处理、指令分发；
4. **应用交互层**：面向用户与外部工具的交互接口。

图 4-1 AgentOS 本地执行层分层架构

图4-1 AgentOS本地执行层分层架构



说明：从下至上展示数据持久层、系统调用层、核心调度层、应用交互层的层级关系与核心组件

4.2 四表分离记忆模型

根据数据的属性、读写模式与用途，将 Agent 的全部记忆拆解到四张逻辑独立的表中：

维度	注册表	环境变量	系统日志	记忆库
核心职责	配置、状态、元指针	运行时参数	行为审计追踪	原始数据“数据湖”
数据结构	树形层级键值对	平铺键值对	追加式时间序列	非结构化文本 / 向量
读写模式	频繁读，低频原子写	频繁读，偶发写	追加写，离线读	低频读，低频写
变更频率	分钟 / 任务级	秒 / 会话级	实时追加	天 / 周级
是否参与推理	是，编译器直接注入	是，编译器注入	否，仅离线审计	按指针被动调度
确定性	极高，原子更	极高	极高，不可变	低，语义模糊

	新			
典型体积	KB 级	Byte 级	MB 级（可轮转）	GB-TB 级
经典 OS 对应	控制寄存器 / 进程控制块	用户态栈上下文	内核审计日志	磁盘文件系统

4.2.1 注册表：确定性元数据库

注册表是 AgentOS 最核心的创新组件。设计灵感源自 Windows 注册表，但针对 Agent 的状态管理需求进行了全面重构。其核心职责是：存储 Agent 和用户的所有确定性、结构化、可快速索引的配置与状态信息。它不存储数据本身，只存储“关于数据的数据”和“当前的运行状态”。

注册表的构建不需要复杂的信息抽取，而是在任务执行过程中由编译器直接进行确定性写入。例如，当 Agent 完成数据清洗步骤后，编译器不是生成一段“我已经完成了数据清洗...”的自然语言，而是原子性地写入

```
HKEY_CURRENT_SESSION\CurrentTask\Phase = "data_cleaning_complete"。
```

技术说明：树形路径查询的延迟与注册表整体大小无关，仅与路径深度（通常≤5层）相关，在工程上可视为常数级确定性操作，而非严格理论意义的 O(1) 复杂度。

注册表 Schema 定义：

```
Plaintext
注册表完整键值空间定义
=====

根键：HKEY_LOCAL_AGENT (HKLA)
描述：Agent 自身的核心配置，不可变或极少变更
权限：内核态可写，用户态可读
子树结构：
  \Identity
    \AgentName: string           // Agent 显示名称
    \AgentID: string             // 全局唯一标识符 (UUID v4)
    \Version: string             // 语义版本号
    \CreatedAt: timestamp        // 创建时间 (ISO 8601)
  \Soul
    \CoreTone: enum              // Professional_Concise |
```

```

Friendly_Verbose | ...
  \MaxResponseLength: integer // 最大响应 Token 数
  \HumorLevel: integer        // 幽默程度 0-5
  \Formality: enum            // Formal | SemiFormal | Casual
  \FullTextPointer: string    // 指向记忆库中完整灵魂文本的指针
  \Capabilities
    \MCP\{mcp_name}
      \Status: enum           // Active | Inactive | Error
      \Path: string           // 可执行路径或端点 URL
      \Version: string
      \RegisteredAt: timestamp
      \CapabilityList: [string] // 该 MCP 提供的具体能力列表
  \Ethics
    \AllowedDomains: [string] // 允许执行的操作域白名单
    \BlockedPatterns: [string] // 禁止的指令模式黑名单

```

根键: HKEY_CURRENT_USER (HKCU)

描述: 当前交互用户的长期信息

权限: 内核态可写, 用户态可读

子树结构:

```

  \Profile
    \Name: string              // 用户名称
    \ExpertiseLevel: enum      // Beginner | Intermediate |
Advanced | Expert
    \Language: string          // 首选语言代码 (zh-CN, en-
US, ...)
    \Timezone: string          // 时区 (Asia/Shanghai, ...)
  \Preferences
    \CodeStyle
      \Language: string        // 首选编程语言
      \Comments: enum          // Verbose | Minimal | None
      \Formatting: enum         // PEP8 | Prettier | Google
  | ...
    \UX
      \Theme: enum             // Dark | Light | Auto
      \OutputFormat: enum       // Markdown | PlainText |
RichText
    \ProjectRegistry
      \{ProjectID}
        \Name: string
        \Path: string          // 本地工作目录绝对路径
        \Status: enum          // Active | Archived | Paused
        \ConfigPointer: string // 指向项目配置的指针

```

```
\CreatedAt: timestamp
\LastOpenedAt: timestamp
```

根键: HKEY_LOCAL_MACHINE (HKLM)

描述: Agent 运行的宿主环境配置

权限: 系统管理员可写

子树结构:

```
\Hardware
  \CPU\Cores: integer
  \GPU\Model: string
  \GPU\MemoryGB: integer
  \RAM\TotalGB: integer
  \Disk\FreeGB: integer
\Software
  \Python\Version: string
  \Python\VirtualEnv: string
  \Node\Version: string
  \Git\Version: string
\Network
  \Status: enum           // Online | Offline | Restricted
  \DefaultAPIEndpoint: string
  \Proxy\Enabled: boolean
  \Proxy\Address: string
```

根键: HKEY_CURRENT_SESSION (HKCS)

描述: 当前会话的易失性状态, 变化最频繁

权限: 内核态可读写, 用户态只读

子树结构:

```
\CurrentTask
  \ID: string             // 任务唯一标识
  \Intent: string         // 任务意图简述
  \Phase: string          // 当前阶段标识
  \Priority: enum          // LOW | NORMAL | HIGH | URGENT
  \Status: enum           // Ready | Running | Blocked |
Suspended
  \CreatedAt: timestamp
  \Checkpoint
    \LastStep: string     // 最后完成步骤描述
    \LastScript: string   // 最后执行的脚本路径
    \OutputHash: string   // 最后输出的 SHA256 哈希
    \SavedAt: timestamp   // 检查点保存时间
  \ActiveEntities
    \{EntityName}
```

```

        \Type: enum                // File | Function | Class |
Service | Person
        \Pointer: string           // 指向实体具体位置的指针
        \LastAccessed: timestamp
    \InteractionState
        \LastUserIntent: string    // 最近一次用户意图分类
        \AwaitingUserInput: boolean
        \TurnCount: integer        // 当前会话交互轮次
        \ContextSummaryPointer: string // 当前会话摘要的指针

根键: HKEY_CLOUD_SYNC (HKCSY)
描述: 云端同步状态配置
权限: 同步服务可写
子树结构:
    \CloudMemory
        \Endpoint: string          // 云内存服务端点 URL
        \SpaceID: string           // 用户云内存空间 ID
        \SyncStatus: enum          // Synced | Syncing | Conflict |
Offline
    \LastSyncTime: timestamp
    \Quota\TotalMB: integer
    \Quota\UsedMB: integer
    \CachedAssets
        \{AssetID}
            \Type: enum             // persistent | skill | context
| temporary
            \TTL: string            // 缓存有效期
            \SyncedAt: timestamp
            \Hash: string           // 资产内容 SHA256

```

4.2.2 环境变量：运行时即时参数

环境变量存储那些需要在运行时被频繁读取、且可能在不同会话间快速切换的工程参数，与 Agent 的核心记忆解耦。

```

Plaintext
环境变量标准定义
=====

# --- 模型调用配置 ---
LLM_API_KEY=sk-*****          # API 密钥（敏感信息，仅存于环境变量）

```

```
LLM_MODEL_NAME=deepseek-coder-v2 # 当前使用的模型名称
LLM_TEMPERATURE=0.1 # 推理温度
LLM_MAX_TOKENS=4096 # 单次最大输出 Token
LLM_TIMEOUT=30 # 调用超时秒数

# --- MCP 工具认证 ---
MCP_SERVER_AUTH_TOKEN=mcp-token-*****

# --- 网络与代理 ---
PROXY_ENABLED=true
PROXY_ADDRESS=127.0.0.1:7890

# --- 运行时环境 ---
WORKING_DIRECTORY=/home/user/agent/workspace
LOG_LEVEL=INFO
```

核心特性：

- **会话级隔离**：不同会话拥有独立的环境变量，切换会话即切换参数；
- **即时生效**：修改环境变量无需改动注册表或提示词，模型切换、参数调整即时生效；
- **敏感信息承载**：API 密钥等仅存于环境变量，不进入注册表与记忆库，避免持久化泄露。

4.2.3 系统日志：不可变行为记录

系统日志是一个追加式、不可变的、按时间戳排序的记录系统。

Plaintext

系统日志条目结构

=====

```
{
  "timestamp": "2024-05-20T14:32:05.123Z", // ISO 8601 毫秒精度
  "event_id": "evt_7f3a2d", // 全局唯一事件 ID
  "event_type": "MCP.INSTALL", // 事件类型（见下表）
  "severity": "INFO", // DEBUG | INFO |
WARN | ERROR | FATAL
  "agent_id": "agent_7f3a", // 执行 Agent ID
  "session_id": "sess_20240520", // 会话 ID
  "task_id": "task_20240520_1455", // 关联任务 ID（可为
null）
  "details": { // 事件详情（结构因类
```



```
型而异)
    "mcp_name": "web_search_plus",
    "version": "1.2.0",
    "installation_path": "/agents/mcps/web_search_plus",
    "registered_capabilities": ["search_web", "fetch_page"]
  },
  "result": "SUCCESS",                                // SUCCESS | FAILURE
| PARTIAL
  "source": "compiler"                                // 日志来源组件
}
```

标准事件类型：

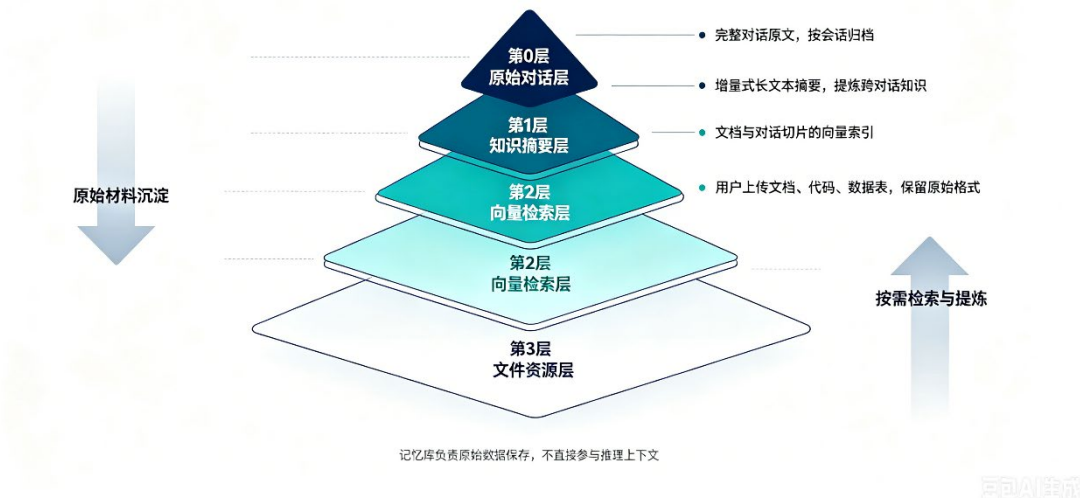
事件类别	事件类型	触发场景
任务类	TASK.START, TASK.PAUSE, TASK.RESUME, TASK.COMPLETE, TASK.ERROR	任务生命周期各阶段
注册表类	REGISTRY.SET, REGISTRY.DELETE, REGISTRY.CONFLICT	注册表键值变更
工具类	MCP.INSTALL, MCP.INVOKE, MCP.FAULT	MCP 工具安装、调用与故障
交互类	USER.INPUT, USER.INTERRUPT, AGENT.RESPONSE	用户交互事件
云端类	CLOUD.SYNC, CLOUD.LOAD, CLOUD.STORE	云端同步与缓存操作

4.2.4 记忆库：非结构化数据湖

记忆库是所有原始、非结构化和半结构化数据的最终栖息地。其内部分层为：

图 4-2 记忆库内部分层架构

图4-2 记忆库内部分层架构



说明：展示记忆库内原始文本层、向量索引层、语义摘要层、指针映射层的四层结构

4.3 操作系统原语与系统调用接口

为实现严格的分层隔离，AgentOS 定义了标准系统调用接口。编译器作为内核态调度器，通过系统调用访问底层资源；上层任务与外部 Agent 只能通过系统调用与系统交互，无法直接操作底层数据。

权限模型：

- 内核态：编译器与系统核心进程，可执行所有系统调用；
- 用户态：业务任务、外部 Agent、第三方插件，仅可执行受限的只读调用与授权操作。

系统调用接口完整定义：

```
Plaintext
系统调用接口定义
=====

/* ===== 注册表操作类 ===== */

// 读取指定路径的注册表键值
// 权限：用户态可读
// 返回：键值（string | number | boolean | object | null）
sys_registry_read(key_path: string) -> Any

// 原子写入指定键值（若父键不存在则自动创建路径）
// 权限：内核态专属
```

```

// 返回：写入成功与否
sys_registry_write(key_path: string, value: Any) -> bool

// 删除指定键及其所有子键
// 权限：内核态专属
// 返回：删除成功与否
sys_registry_delete(key_path: string) -> bool

// 监听指定键值的变化
// 权限：用户态可调用
// 参数：
//   event_types: 监听的事件类型，可选["SET", "DELETE", "CONFLICT"]
// 返回：监听器 ID，用于取消监听
sys_registry_watch(key_path: string, event_types: array, callback:
function) -> string

/* ===== 环境变量操作类 ===== */

// 获取指定环境变量值
// 权限：用户态可读
sys_env_get(var_name: string) -> string

// 设置当前会话环境变量
// 权限：用户态可修改当前会话
sys_env_set(var_name: string, value: string) -> bool

// 列出当前所有环境变量
// 权限：用户态可读
sys_env_list() -> dict

/* ===== 日志操作类 ===== */

// 追加一条日志记录
// 权限：用户态可追加
// 返回：事件 ID
sys_log_append(event_type: string, details: dict, severity: string
= "INFO") -> string

// 按条件查询日志
// filter: {event_type, severity, task_id, time_range: [start,

```

```

end]]}
// 权限: 用户态可查询
sys_log_query(filter: dict, limit: int = 100) -> list

/* ===== 记忆库操作类 ===== */

// 根据指针加载记忆库数据
// 权限: 用户态可调用
sys_memory_load(pointer: string, params: dict = {}) -> Any

// 存储数据到记忆库, 返回指针
// 权限: 内核态可写入
sys_memory_store(data: Any, metadata: dict = {}) -> string

// 语义检索记忆库
// 权限: 用户态可调用
// 返回: [{content, score, pointer}, ...]
sys_memory_search(query: string, top_k: int = 5, filters: dict =
{}) -> list

/* ===== 进程管理类 ===== */

// 创建新任务进程, 初始化 PCB 并写入注册表
// 权限: 内核态专属
// 返回: task_id
sys_process_create(intent: string, params: dict, priority: string
= "NORMAL") -> string

// 控制进程状态
// action: PAUSE | RESUME | TERMINATE
// 权限: 内核态专属
sys_process_control(task_id: string, action: string) -> bool

// 获取进程当前状态
// 权限: 用户态可调用
sys_process_status(task_id: string) -> dict

// 列出所有活跃进程
// 权限: 用户态可调用
sys_process_list() -> list

```

```

/* ===== 工具调用类 ===== */

// 调用指定 MCP 工具
// 权限：用户态可调用（需在注册表能力清单中已注册）
sys_tool_invoke(mcp_id: string, tool_name: string, params: dict) -
> dict

```

4.4 进程管理模型

AgentOS 将单个用户任务抽象为 **Agent 进程**，借鉴经典操作系统的进程管理思想，实现任务的全生命周期管理。

进程控制块（PCB）结构：

Plaintext

进程控制块（PCB）定义

=====

每个 Agent 进程对应一个 PCB，其完整状态存储在注册表
HKEY_CURRENT_SESSION\CurrentTask 路径下。

```

PCB = {
  // --- 进程标识 ---
  task_id: string,           // 全局唯一任务 ID (UUID v4)
  session_id: string,        // 所属会话 ID
  created_at: timestamp,     // 创建时间
  created_by: string,        // 创建者（用户/调度器）

  // --- 进程状态 ---
  state: enum {              // 当前状态
    READY,                   // 就绪，等待调度
    RUNNING,                 // 运行中
    BLOCKED,                 // 阻塞，等待 I/O 或用户输入
    SUSPENDED,               // 挂起，可恢复
    TERMINATED               // 终止
  },
  phase: string,             // 当前任务阶段
  priority: enum { LOW, NORMAL, HIGH, URGENT },

  // --- 资源上下文 ---
  env_snapshot: dict,        // 进程创建时的环境变量快照

```

```

active_entities: [string], // 活跃实体指针列表
checkpoint: { // 检查点信息
    last_step: string, // 最后完成步骤
    data_snapshot: dict, // 中间数据快照
    saved_at: timestamp
},

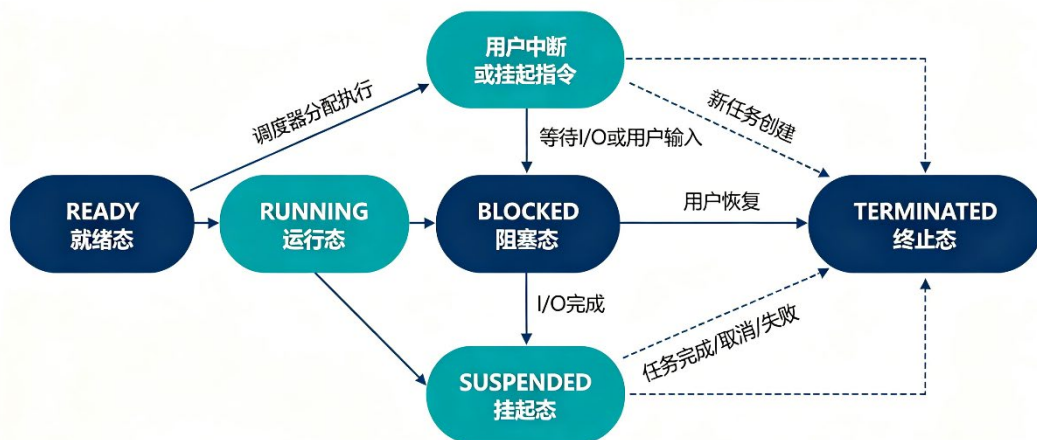
// --- 统计信息 ---
stats: {
    total_tokens_used: integer, // 累计 Token 消耗
    tool_calls: integer, // 工具调用次数
    wall_time_ms: integer // 运行时长
}
}

```

进程状态机：

图 4-3 Agent 进程状态转换图

图4-3 Agent进程状态转换图



抢占式优先级调度 | 阻塞态释放调度资源

图4-3

说明：展示 *READY*、*RUNNING*、*BLOCKED*、*SUSPENDED*、*TERMINATED* 五种状态之间的转换条件与触发事件

调度策略：编译器作为调度器，采用基于优先级的抢占式调度。高优先级任务（如用户即时指令）可抢占低优先级后台任务；同优先级按先来先服务原则执行；阻塞态任务释放调度资源，等待事件触发后重新进入就绪队列。

4.5 中断与异常处理

AgentOS 建立了完整的中断处理机制，所有异常事件均被定义为标准中断，由统一的

中断处理程序接管，而非依赖模型自行处理。

中断向量表：

中断码	中断名称	触发场景	优先级
INT_USER_OVERRIDE	用户打断中断	用户中途打断、修改指令、切换任务	最高
INT_TOOL_FAULT	工具故障中断	MCP 调用超时、执行失败、返回格式错误	中高
INT_RESPONSE_ERROR	响应异常中断	模型返回不符合工单 Schema、内容违规	中
INT_CONCURRENCY_CONFLICT	并发冲突中断	注册表并发写入冲突、云同步状态不一致	低

中断处理程序伪代码：

Python

中断处理程序参考实现

```
def handle_interrupt(int_code: str, context: dict) -> dict:
    # 1. 保存当前进程现场
    save_checkpoint(current_task_id)

    # 2. 根据中断码分派处理
    match int_code:
        case "INT_USER_OVERRIDE":
            # 用户打断：挂起当前任务，解析新意图
            sys_process_control(current_task_id, "SUSPEND")
            new_intent = compiler.parse(user_input)
            if new_intent.overwrites_current:
                sys_process_control(current_task_id, "TERMINATE")
            new_task_id = sys_process_create(new_intent)
            return {"action": "SWITCH", "new_task_id":
new_task_id}
```



```

        case "INT_TOOL_FAULT":
            # 工具故障：根据策略决定重试/降级/失败
            error_info = context.error
            retry_config =
sys_registry_read("HKLA\\Capabilities\\MCP\\"
                    + context.mcp_id + "\\RetryPolicy")
            if retry_config and context.retry_count <
retry_config.max_retries:
                # 指数退避重试
                wait(retry_config.base_delay * (2 ^
context.retry_count))
                return {"action": "RETRY", "retry_count":
context.retry_count + 1}
            elif context.fallback_mcp:
                # 降级到备用工具
                return {"action": "FALLBACK", "fallback_mcp":
context.fallback_mcp}
            else:
                # 失败，通知用户
                return {"action": "FAIL", "error": error_info}

        case "INT_RESPONSE_ERROR":
            # 响应异常：要求模型重新生成或切换备用模型
            if context.retry_count < 3:
                return {"action": "RETRY_WITH_FEEDBACK",
                        "feedback": "Response schema mismatch"}
            else:
                # 切换备用模型重试
                sys_env_set("LLM_MODEL_NAME", fallback_model)
                return {"action": "RETRY_WITH_FALLBACK_MODEL"}

        case "INT_CONCURRENCY_CONFLICT":
            # 并发冲突：LWW 策略，以时间戳最新为准
            local_ts = sys_registry_read(conflict_key +
"\\_timestamp")
            remote_ts = context.remote_timestamp
            if local_ts >= remote_ts:
                return {"action": "KEEP_LOCAL"}
            else:
                sys_registry_write(conflict_key,
context.remote_value)
                return {"action": "ACCEPT_REMOTE"}

```

3. 记录中断事件到日志

```
sys_log_append("INTERRUPT." + int_code, context)
```

```
# 4. 恢复进程现场或返回处理结果
```

```
return {"action": "COMPLETE"}
```

5. 编译通信层：工单协议与编译器实现

如果说本地记忆层赋予了 Agent 结构化的自我认知，那么编译通信层则定义了 Agent 与世界交互的语言。本章详细阐述两级编译器架构、工单协议完整规范、认知资产缓存握手协议，以及与 RAG 的互补机制。

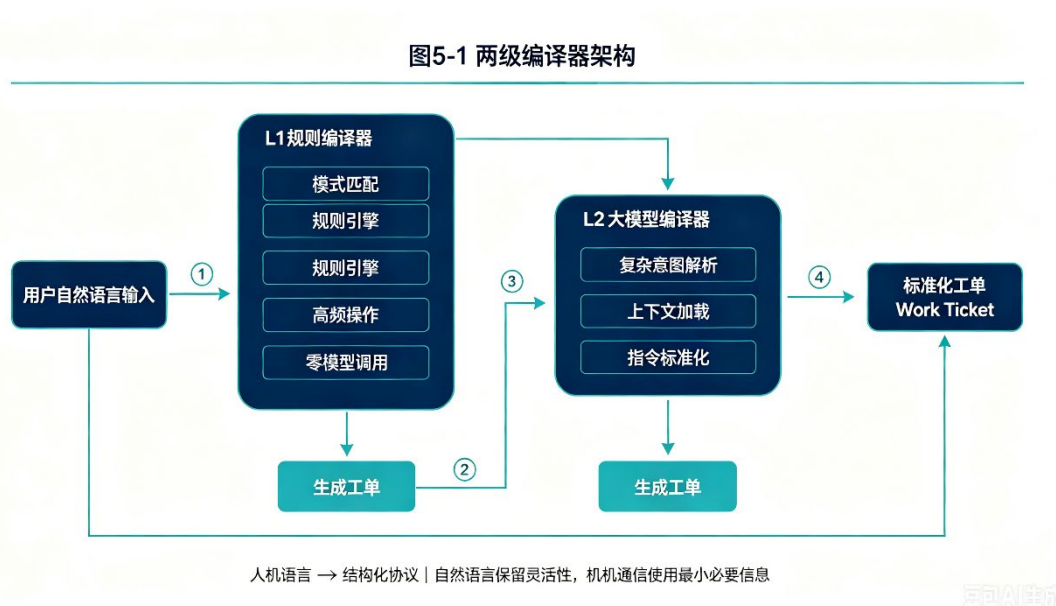
5.1 编译器核心定位与两级架构

编译器是 AgentOS 的核心网关与翻译官，承上启下连接用户、本地系统与云端模型。其五大核心职能为：意图净化（剥离自然语言的模糊与冗余）、状态加载（查询注册表与环境变量）、工单生成（打包为标准化工单）、结果解析（接收并解析结构化响应）、响应翻译（将结构化结果转化为自然语言）。

编译器本质上是人机语言与机机语言之间的翻译器，也是上下文污染的第一道防火墙。所有进出系统的信息，都必须经过编译器的过滤与转换。

两级编译器实现方案：

图 5-1 两级编译器架构



说明：展示 L1 规则编译器与 L2 大模型编译器的流水线，包含意图输入、规则匹配、失败降级、工单生成的完整流程

L1 规则编译器完整参考实现:

Python

L1 规则编译器完整参考实现

处理高频、确定性操作，零模型调用成本

```
import re
from typing import Dict, Optional

class L1RuleCompiler:
    """基于规则引擎的轻量编译器，处理确定性操作"""

    def __init__(self):
        # 注册意图匹配规则（按优先级排序）
        self.rules = [
            {
                "pattern": r"^s*切换(?:到|成)?模型\s*(?P<model>\S+)\s*$",
                "handler": self._handle_switch_model,
                "priority": 10
            },
            {
                "pattern": r"^s*(?:暂停|停下|等一下|先停)\s*$",
                "handler": self._handle_pause_task,
                "priority": 9
            },
            {
                "pattern": r"^s*(?:继续|恢复)(?:之前的|上次的)?(?:任务|工作)?\s*$",
                "handler": self._handle_resume_task,
                "priority": 8
            },
            {
                "pattern": r"^s*(?:查看|显示|查询)(?:当前)?(?:状态|进度)\s*$",
                "handler": self._handle_query_status,
                "priority": 7
            },
            {
                "pattern": r"^s*设置\s*(?P<key>\S+)\s*为\s*(?P<value>\S+)\s*$",
                "handler": self._handle_set_config,
                "priority": 6
            }
        ]
```

```

    },
    {
        "pattern": r"^\\s*(?:清空|清除)当前会话\\s*$",
        "handler": self._handle_clear_session,
        "priority": 5
    }
]
# 按优先级排序规则
self.rules.sort(key=lambda x: x["priority"], reverse=True)

def try_compile(self, user_input: str) -> Optional[Dict]:
    """尝试匹配规则，成功返回工单，失败返回 None"""
    cleaned_input = user_input.strip()
    for rule in self.rules:
        match = re.match(rule["pattern"], cleaned_input,
re.IGNORECASE)
        if match:
            return rule["handler"](match.groupdict())
    return None # 匹配失败，流转到 L2 大模型编译器

def _handle_switch_model(self, params: Dict) -> Dict:
    model_name = params["model"]
    sys_env_set("LLM_MODEL_NAME", model_name)
    sys_log_append("ENV.SET",
                    {"var": "LLM_MODEL_NAME", "value":
model_name})
    return {
        "header": {
            "version": "1.0",
            "agent_id":
sys_registry_read("HKLA\\Identity\\AgentID"),
            "session_id":
sys_registry_read("HKCS\\SessionID"),
            "message_id": f"msg_{int(time.time()*1000)}",
            "timestamp": datetime.utcnow().isoformat() + "Z",
            "msg_type": "CONTROL"
        },
        "instruction": {
            "op": "SWITCH_MODEL",
            "model": model_name
        },
        "cache_load": []
    }
}

```

```

def _handle_pause_task(self, params: Dict) -> Dict:
    current_task = sys_registry_read("HKCS\\CurrentTask\\ID")
    if current_task:
        sys_process_control(current_task, "PAUSE")
        sys_log_append("TASK.PAUSE", {"task_id":
current_task})
    return {
        "header": {
            "version": "1.0",
            "agent_id":
sys_registry_read("HKLA\\Identity\\AgentID"),
            "session_id":
sys_registry_read("HKCS\\SessionID"),
            "message_id": f"msg_{int(time.time()*1000)}",
            "timestamp": datetime.utcnow().isoformat() + "Z",
            "msg_type": "CONTROL"
        },
        "instruction": {
            "op": "PAUSE_TASK",
            "task_id": current_task
        }
    }

def _handle_resume_task(self, params: Dict) -> Dict:
    last_task = sys_registry_read("HKCS\\CurrentTask\\ID")
    checkpoint =
sys_registry_read("HKCS\\CurrentTask\\Checkpoint")
    if last_task:
        sys_process_control(last_task, "RESUME")
        sys_log_append("TASK.RESUME", {"task_id": last_task})
    return {
        "header": {
            "version": "1.0",
            "agent_id":
sys_registry_read("HKLA\\Identity\\AgentID"),
            "session_id":
sys_registry_read("HKCS\\SessionID"),
            "message_id": f"msg_{int(time.time()*1000)}",
            "timestamp": datetime.utcnow().isoformat() + "Z",
            "msg_type": "TASK_EXECUTE"
        },
        "instruction": {
            "op": "RESUME_TASK",
            "task_id": last_task,

```

```

        "from_checkpoint": checkpoint
    },
    "cache_load": [
        {"id":
sys_registry_read("HKCSY\\CachedAssets\\soul_v2\\ID"),
        "type": "persistent", "required": True}
    ]
}

def _handle_query_status(self, params: Dict) -> Dict:
    task_id = sys_registry_read("HKCS\\CurrentTask\\ID")
    phase = sys_registry_read("HKCS\\CurrentTask\\Phase")
    stats = sys_process_status(task_id) if task_id else {}
    return {
        "header": {
            "version": "1.0",
            "agent_id":
sys_registry_read("HKLA\\Identity\\AgentID"),
            "session_id":
sys_registry_read("HKCS\\SessionID"),
            "message_id": f"msg_{int(time.time()*1000)}",
            "timestamp": datetime.utcnow().isoformat() + "Z",
            "msg_type": "TASK_EXECUTE"
        },
        "instruction": {
            "op": "QUERY_STATUS",
            "task_id": task_id,
            "current_phase": phase,
            "stats": stats
        }
    }

def _handle_set_config(self, params: Dict) -> Dict:
    key, value = params["key"], params["value"]
    config_path = f"HKCU\\Preferences\\{key}"
    sys_registry_write(config_path, value)
    sys_log_append("REGISTRY.SET", {"key": config_path,
"value": value})
    return {
        "header": {
            "version": "1.0",
            "agent_id":
sys_registry_read("HKLA\\Identity\\AgentID"),
            "session_id":

```

```

sys_registry_read("HKCS\\SessionID"),
    "message_id": f"msg_{int(time.time()*1000)}",
    "timestamp": datetime.utcnow().isoformat() + "Z",
    "msg_type": "CONTROL"
},
"instruction": {
    "op": "SET_CONFIG",
    "key": config_path,
    "value": value
}
}

def _handle_clear_session(self, params: Dict) -> Dict:
    # 清空当前会话状态，保留核心配置
    sys_registry_delete("HKCS\\CurrentTask")
    sys_registry_delete("HKCS\\ActiveEntities")
    sys_registry_write("HKCS\\InteractionState\\TurnCount", 0)
    sys_log_append("SESSION.CLEAR", {})
    return {
        "header": {
            "version": "1.0",
            "agent_id":
sys_registry_read("HKLA\\Identity\\AgentID"),
            "session_id":
sys_registry_read("HKCS\\SessionID"),
            "message_id": f"msg_{int(time.time()*1000)}",
            "timestamp": datetime.utcnow().isoformat() + "Z",
            "msg_type": "CONTROL"
        },
        "instruction": {
            "op": "CLEAR_SESSION"
        }
    }
}

```

5.2 工单协议完整规范

工单（Work Ticket）是 Agent 与云端模型、Agent 与 Agent 之间交互的唯一通信单元。工单是结构化的协议文件，而非自然语言文本。

工单协议完整 **Schema（JSON Schema v1.0）**：

```

JSON
{
    // 符合 JSON Schema Draft 07 规范，用于 IDE 语法校验与自动化生成工具

```



```
"$schema": "http://json-schema.org/draft-07/schema#",
"title": "AgentOS Work Ticket Schema v1.0",
"description": "AgentOS 标准化工单协议—机机通信的唯一结构化载体",
"type": "object",
"required": ["header", "instruction"],
"properties": {
  "header": {
    "type": "object",
    "description": "消息元数据—用于路由、鉴权与幂等控制",
    "required": ["version", "agent_id", "session_id",
"message_id", "timestamp", "msg_type"],
    "properties": {
      "version": {
        "type": "string",
        "const": "1.0",
        "description": "协议版本号，用于兼容性检查"
      },
      "agent_id": {
        "type": "string",
        "description": "发送方 Agent 全局唯一标识"
      },
      "session_id": {
        "type": "string",
        "description": "当前会话 ID，用于关联多轮交互"
      },
      "message_id": {
        "type": "string",
        "description": "本消息唯一标识，用于幂等控制与回调匹配"
      },
      "timestamp": {
        "type": "string",
        "format": "date-time",
        "description": "消息创建时间，ISO 8601 格式"
      },
      "priority": {
        "type": "string",
        "enum": ["LOW", "NORMAL", "HIGH", "URGENT"],
        "default": "NORMAL",
        "description": "消息优先级，影响调度顺序"
      },
      "msg_type": {
        "type": "string",
        "enum": ["TASK_EXECUTE", "CONTROL", "CACHE_OP",
"CALLBACK", "DELEGATE"],
```

```

        "description": "消息类型：任务执行/控制指令/缓存操作/回调/任务委派"
    }
}
},
"auth": {
    "type": "object",
    "description": "认证信息—敏感凭证通过环境变量引用，不出现在工单正文中",
    "properties": {
        "api_key_env_ref": {
            "type": "string",
            "description": "API 密钥的环境变量引用名，如'LLM_API_KEY'"
        },
        "user_context_id": {
            "type": "string",
            "description": "用户上下文 ID，用于云端用户隔离"
        },
        "signature": {
            "type": "string",
            "description": "消息签名，用于防篡改校验"
        }
    }
},
"cache_load": {
    "type": "array",
    "description": "本次推理需要加载的云端缓存资产列表",
    "items": {
        "type": "object",
        "required": ["id", "type"],
        "properties": {
            "id": {
                "type": "string",
                "description": "缓存资产唯一 ID，由首次 cache_store 返回"
            },
            "type": {
                "type": "string",
                "enum": ["persistent", "skill", "context", "temporary"],
                "description": "资产类型：persistent=核心灵魂(永久)/skill=技能包/context=项目上下文/temporary=临时"
            }
        },
        "required": {

```

```

        "type": "boolean",
        "default": true,
        "description": "是否必须加载成功，若为 true 且加载失败则任
务失败"
    },
    "version_hint": {
        "type": "string",
        "description": "期望的资产版本，用于缓存一致性校验"
    }
}
},
"instruction": {
    "type": "object",
    "description": "核心执行指令—模型推理的唯一依据",
    "required": ["op"],
    "properties": {
        "op": {
            "type": "string",
            "description": "标准操作码，如 ANALYZE_DATA | WRITE_CODE
| RESUME_TASK | DEBUG_TASK | QUERY_STATUS"
        },
        "task_id": {
            "type": "string",
            "description": "关联任务 ID，用于状态追踪与日志关联"
        },
        "execution_context": {
            "type": "object",
            "description": "执行上下文—数据指针、脚本指针、检查点哈希等
",
            "properties": {
                "data_pointer": {"type": "string"},
                "script_pointer": {"type": "string"},
                "last_checkpoint_hash": {"type": "string"},
                "environment_requirements": {"type": "object"}
            }
        },
        "constraints": {
            "type": "object",
            "description": "执行约束—超时、步数限制等",
            "properties": {
                "max_steps": {"type": "integer"},
                "timeout_seconds": {"type": "integer"},
                "allowed_tools": {"type": "array", "items": {"type":

```

```
"string"}}
    }
  },
  "params": {
    "type": "object",
    "description": "操作相关的额外参数"
  }
}
},
"new_delta": {
  "type": "object",
  "description": "本次交互新增的增量信息，以最小必要为原则",
  "properties": {
    "user_feedback_summary": {
      "type": "string",
      "description": "用户反馈的摘要，而非完整原文"
    },
    "new_log_refs": {
      "type": "array",
      "items": {"type": "string"},
      "description": "新增日志的事件 ID 引用列表"
    },
    "attachments": {
      "type": "array",
      "description": "需要临时传输的附件指针（非原始文件）"
    }
  }
}
},
"cache_store": {
  "type": "array",
  "description": "请求云端持久化的新缓存资产列表",
  "items": {
    "type": "object",
    "required": ["id", "data"],
    "properties": {
      "id": {
        "type": "string",
        "description": "请求的缓存资产唯一 ID"
      },
      "data": {
        "type": ["object", "string"],
        "description": "需要缓存的结构化数据"
      }
    },
    "ttl": {
```

```

        "type": "string",
        "default": "1h",
        "description": "存活时间，支持 s/m/h/d/w 后缀，或
'permanent' 表示永久"
    },
    "overwrite": {
        "type": "boolean",
        "default": false,
        "description": "是否允许覆盖已存在的同名缓存"
    },
    "type": {
        "type": "string",
        "enum": ["persistent", "skill", "context",
"temporary"],
        "description": "缓存资产类型"
    }
}
}
}
}
}
}
}

```

响应报文完整规范：

```

JSON
{
  "header": {
    "request_id": "msg_0000334",
    "status": "SUCCESS",
    "error_code": "SUCCESS",
    "error_message": "",
    "timestamp": "2024-05-20T14:35:05Z",
    "processing_time_ms": 823
  },
  "execution_result": {
    "findings": {
      "root_cause": "Script logic error at line 23.",
      "affected_rows": 150,
      "confidence": 0.95
    },
    "suggested_registry_updates": [
      {"key": "HKCS\\CurrentTask\\Phase", "value":
"error_analysis_complete"},
      {"key": "HKCS\\CurrentTask\\Error", "value":

```

```

"null_handling_logic_error"}
],
"suggested_cache_updates": [
  {"id": "debug_session_20240520", "data": {"breakpoints":
["line_23"]}, "ttl": "1h"}
],
"tool_calls": []
},
"response_to_user": "找到了问题所在。clean_script.py 第 23 行的逻辑
有误，导致约 150 行的缺失值被错误填充。需要我帮你修正脚本并重新运行清洗
吗？",
"cache_updates": [
  {"id": "debug_session_20240520", "action": "STORED",
"expires_at": "2024-05-20T15:35:05Z"}
]
}

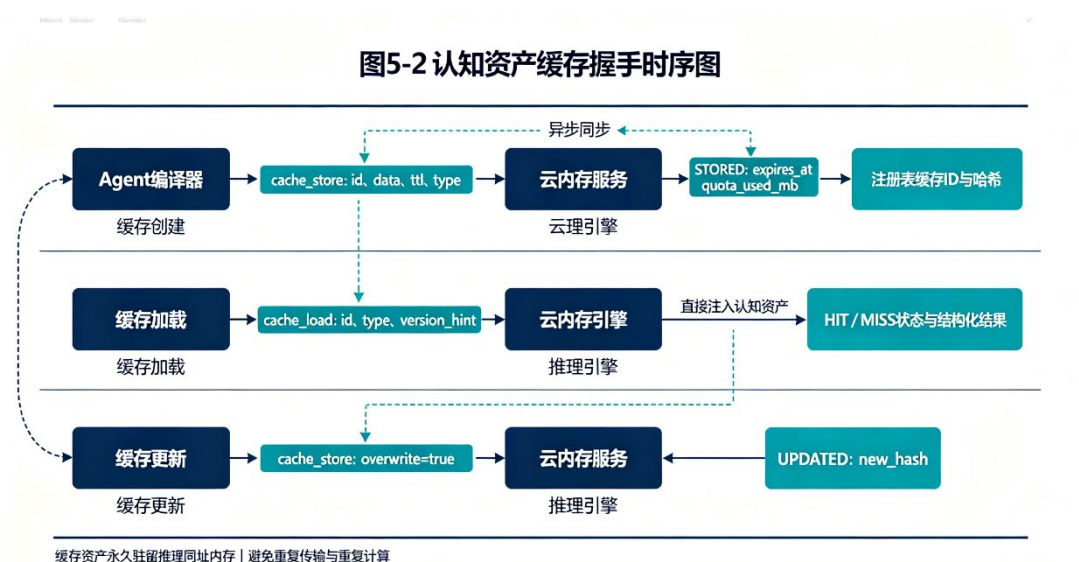
```

5.3 认知资产缓存握手协议

`cache_load` 与 `cache_store` 共同构成了 AgentOS 的认知资产缓存协议，实现了对云端上下文的显式、可编程控制。

缓存握手完整流程：

图 5-2 认知资产缓存握手时序图



说明：展示编译器、云内存服务、推理引擎三方之间 `cache_store`、`cache_load`、资产注入、结果返回的时序交互

5.4 与 RAG 的互补机制

AgentOS 并非淘汰 RAG，而是将其从“核心记忆主干”降级为“记忆库的检索组件”。

Plaintext

RAG 在 AgentOS 中的定位

=====

传统架构：用户输入 → RAG 检索 → 片段直接注入上下文 → 模型推理

AgentOS： 用户输入 → 编译器 → `sys_memory_search` → RAG 检索记忆库

→ 编译器提炼检索结果 → 关键信息写入注册表

→ 原始片段释放 → 工单（仅含提炼后的结构化信息）→ 模型推理

完整调用链路：

1. 任务执行中，编译器判断需要调用历史知识；
2. 编译器通过 `sys_memory_search` 向记忆库发起检索请求；
3. 记忆库内部 RAG 引擎执行向量检索，返回 Top-K 相关片段；
4. 编译器对检索结果进行提炼，提取关键结论与结构化信息；
5. 提炼后的信息更新到注册表对应字段，或整合进工单的 `new_delta`；
6. 原始检索片段被释放，不进入最终的推理上下文。

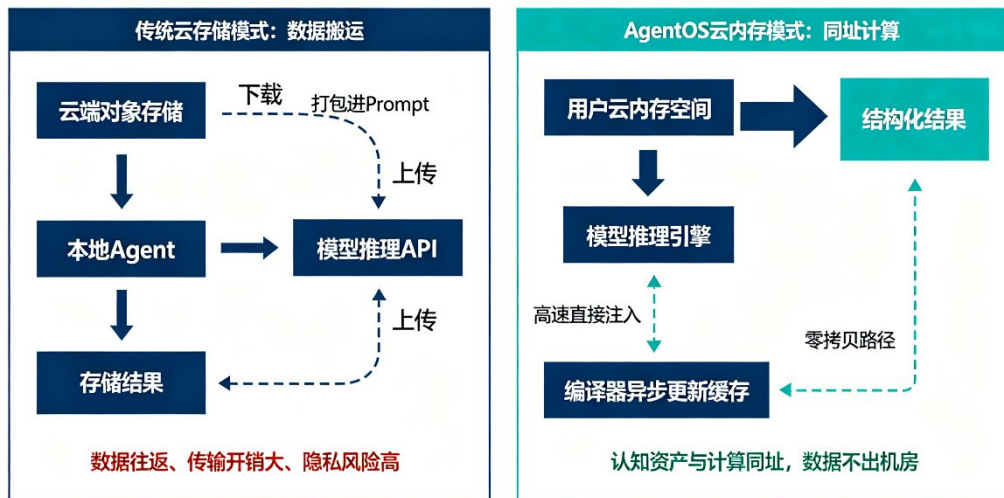
这既保留了 RAG 强大的语义检索能力，又避免了其直接注入上下文带来的污染与臃肿问题。

6. 云端云内存架构

6.1 范式跃迁：从云存储到云内存

当前模式是“云存储”模式：Agent 技能、记忆放在云端对象存储（如 S3），启动时下载到本地，执行时打包进 Prompt 上传。数据在本地与云端之间往返。AgentOS 的“云内存”模式彻底重构了这一链路：认知资产直接存储在推理引擎同址的专用内存空间中，推理时直接从相邻内存分区读取、注入上下文。

图6-1 从云存储到云内存



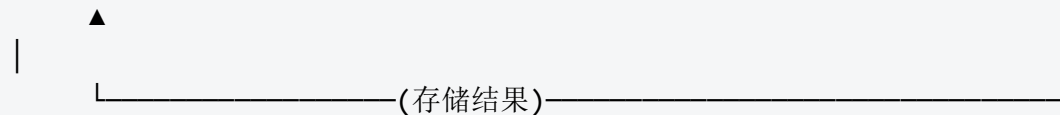
两种模式对比：

Plaintext

传统云存储模式（数据搬运）

=====

[云端对象存储] —(下载)—> [本地 Agent] —(打包进 Prompt)—> [模型推理 API]

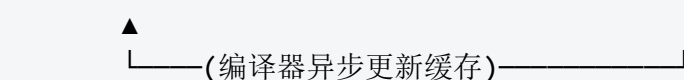


问题：数据在云端与本地间往返，传输开销大，隐私风险高

AgentOS 云内存模式（同址计算）

=====

[用户云内存空间] —(高速直接注入)—> [模型推理引擎] —(推理)—> [结构化结果]



优势：认知资产与计算同址，数据不出机房，纳秒级注入

6.2 三层落地路径

层级	名称	技术实现	典型延迟	适用阶段
L1	软件定义云内	基于现有 KV	10-100 毫秒	即刻落地

	存	缓存 API 封装, SSD 持久化 + 内存热数据		
L2	近推理内存池	RDMA 高速内存池, 常驻内存, 绕过业务网关	1-10 微秒	规模用户
L3	同址内存融合	GPU HBM 显存分区持久化, MMU 直接寻址, 零拷贝	10-100 纳秒	终极形态

6.3 CAP 权衡与一致性模型

- **本地注册表：强一致性（CP）。**是状态的唯一真值源，任何更新必须原子写入本地。
- **云端云内存：最终一致性（AP）。**作为异步镜像与性能加速层，冲突时始终以本地状态为准。

同步机制：

Plaintext

本地-云端状态同步协议

=====

1. 本地状态变更 → `sys_log_append("REGISTRY.SET", ...)` → 异步发送同步请求
2. 每次工单请求携带本地状态哈希 (`state_digest`)
3. 云端校验: `if cloud.cache_digest != request.state_digest` → 触发全量同步
4. 冲突解决: LWW 策略，以时间戳最新为准，本地优先

7. 安全与隐私模型

7.1 威胁模型定义

攻击者能力假设：可窃取云端存储数据（拖库攻击）、可监听公网传输链路（中间人攻击）、可通过社工或漏洞获取用户的工单数据、可尝试篡改编译器或注入恶意指令。

不假设攻击者可以：物理攻破用户本地终端并获取最高权限、用户主动泄露敏感数据、攻破底层加密算法本身。

7.2 三大分离原则

原则一：配置与材料分离。云端只存储编译后的配置与技能（How），不存储原始隐私材料（What）。即使云内存被拖库，攻击者只能看到“用户习惯用 Python 写详细注释”这类元信息。

原则二：指针与数据分离。注册表中存储的是文件路径、变量名、哈希值等指针，而非数据本身。黑客拿到注册表，只能得到 `C:\...\秘密计划.docx` 这样的路径字符串，无法穿透到用户本地获取真实文件——就像拿到了图书馆的索引卡片柜，离真正读到书的内容还有物理世界的鸿沟。

原则三：陪伴与执行分离。编译器将情感陪伴请求与任务执行请求分流至不同模型。执行模型永远接触不到自然语言情感内容，从源头杜绝情感污染。

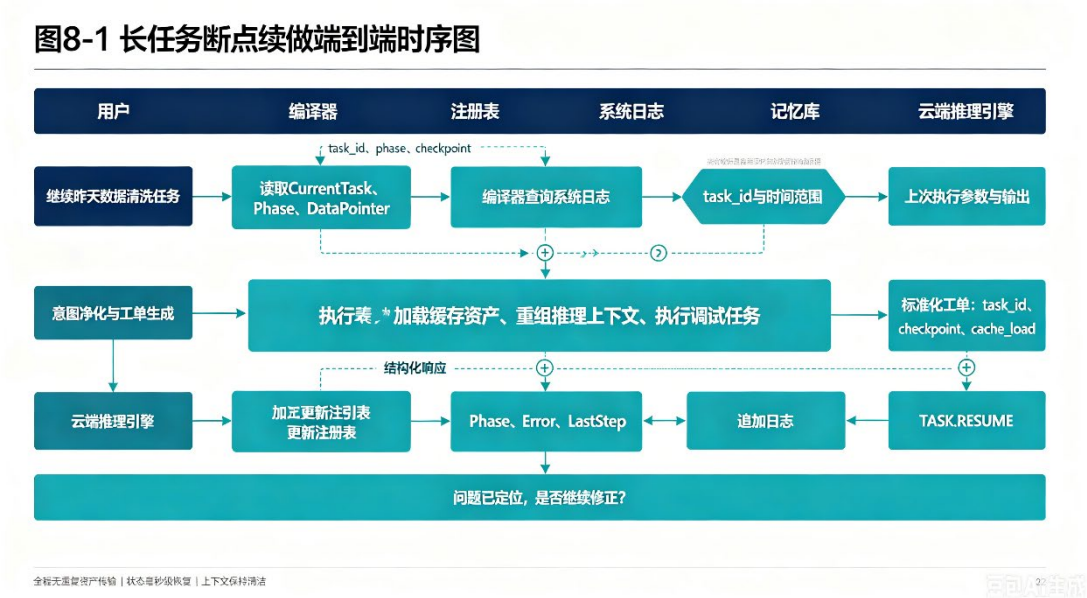
7.3 风险缓解方案

风险	描述	缓解方案
指针侧信道	指针访问模式泄露行为规律	访问扰动（插入随机噪声）、批量聚合请求、流量填充
编译器投毒	编译器被篡改，伪造工单	哈希校验（注册表关键键值附加 SHA256）、操作留痕审计、核心配置变更需用户二次确认
临时数据泄露	任务执行时材料临时注入沙箱	沙箱隔离（每任务独立沙箱）、用后即焚（执行完毕内存清零）、差分隐私噪声（仅适用于统计分析类任务，代码生成等精确计算场景禁用）

8. 端到端 workflow

以“继续昨天数据清洗任务”为例，展示完整时序流程：

图 8-1 长任务断点续做端到端时序图



9. 实验评估设计

9.1 基线选择

基线编号	方案类型	具体实现
基线 1	纯长上下文	128K 上下文窗口，完整对话历史直接拼接
基线 2	RAG 增强	向量数据库存储对话切片，Top-K=5 检索注入上下文
基线 3	虚拟上下文（MemGPT 风	分页式记忆管理，模型自

	格)	主调度记忆换入换出
--	----	-----------

9.2 数据集与指标

采用 **AgentBench** 标准基准 [21] 加自定义 **30 轮长任务序列**（涵盖多步骤数据处理、代码开发调试、任务切换与中断恢复、穿插 5 轮闲聊干扰）。

指标类别	具体指标	测量方法
效率	单轮平均输入 Token 数、Token 增长斜率、P95 端到端延迟	统计每轮 API 调用的输入 Token 数，线性回归计算斜率
准确性	状态恢复准确率、确定性信息回忆 F1 值、长任务完成率	对比恢复后的状态与真实状态，计算精确匹配率
鲁棒性	抗污染衰减率、异常处理成功率	插入干扰后任务完成率相对于无干扰基线的下降比例
成本	总 Token 消耗、单位任务成本	汇总完整 30 轮任务序列的输入输出 Token 总数

9.3 消融实验设计

消融版本	移除组件	实现方式	验证目标
A1	移除注册表	状态信息改为通过上下文携带	验证注册表对状态确定性与效率的贡献
A2	移除编译器	直接用自然语言提示与模型交互	验证编译通信对防污染与效率的贡献
A3	移除云内存	每次重传完整资产，不使用缓存	验证云内存对传输效率与成本的贡献

A4	AgentOS 完整版	全部组件启用	基准值
----	-------------	--------	-----

预期结果：AgentOS 的 Token 消耗保持平稳，不随轮次增长；状态恢复准确率接近 100%；抗污染衰减小于 5%；消融各组件后性能出现不同程度退化，验证三位一体架构的协同效应。

10. 产业落地路径

10.1 厂商阶梯式落地路径

- **第一阶段（0-6 个月）：可编程缓存 API。**基于现有 KV 缓存能力封装开放 API，实现缓存持久化与按空间计费，完成核心技术验证。
- **第二阶段（6-18 个月）：工单协议与编译器 SDK。**发布协议规范与 SDK，推出开源运行时参考实现，建立开发者社区，形成协议生态。
- **第三阶段（18-36 个月）：全托管 AgentOS 云服务。**推出覆盖本地到云端的完整解决方案，上线多 Agent 协作网络与技能市场，完成生态闭环。

10.2 对大模型厂商的战略价值

若率先落地 AgentOS 架构，将获得三重差异化优势：

- **成本端：**大幅降低重复 Token 传输与计算，提升单卡吞吐，在价格战中构建成本护城河。
- **生态端：**通过云内存与工单协议建立开发者粘性，跳出同质化的模型参数竞赛——用户迁移成本从“换一个 API Key”变为“放弃半年积累的认知资产”。
- **企业级市场：**架构级隐私方案精准满足政企客户的数据合规要求，打开高价值 ToB 市场。

11. 多 Agent 协作与 Agent 互联网展望

将工单协议扩展为 Agent 间的通用通信语言，支持任务委派、结果回调与服务发现。当足够多的 Agent 接入统一的工单协议，一个全新的 **Agent 互联网**将应运而生。工单协议是其“TCP/IP”，Agent 服务目录是其“DNS”，技能市场是其“App Store”。不同领域、不同厂商的 Agent 能力可以像乐高积木一样自由组合，形成强大的网络效应。

12. 结论

我们正站在 Agent 发展的范式十字路口。一条路是继续在应用层用精巧的工程手段修补“上下文即记忆”的固有缺陷，承受效率的线性损耗与复杂度的指数增长。另一条路，是本文提出的 AgentOS 之路——回归计算本质，从系统架构层面进行重构。

本文提出的 AgentOS，以注册表、编译器、云内存三位一体设计，系统性回应了当前 Agent 在记忆、通信、安全和商业模式上的核心挑战。它不是一个更好的记忆管理器，而是一套更正确的计算哲学在智能体领域的首次系统化实践。

我们的目标，是让 Agent 从一个每次醒来都需要阅读长篇日记来知道自己是谁的“失忆症患者”，变成一个拥有结构化自我、高效通信与主权隐私的“数字大脑”。这，才是通用人工智能得以可靠运行的基础设施。

参考文献

- [1] Park J S, O'Brien K, Cai C J, et al. Generative Agents: Interactive Simulacra of Human Behavior[C]//Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology. 2023: 1-22.
- [2] Packer C, Gao S, Wooders S, et al. MemGPT: Towards LLMs as Operating Systems[J]. arXiv preprint arXiv:2310.08560, 2023.
- [3] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[J]. Advances in Neural Information Processing Systems, 2020, 33: 9459-9474.
- [4] Vaswani A, Shazeer N, Parmar N, et al. Attention is All You Need[J]. Advances in Neural Information Processing Systems, 2017, 30.
- [5] Ritchie D M, Thompson K. The UNIX Time-sharing System[J]. Communications of the ACM, 1974, 17(7): 365-375.
- [6] Solomon D A, Russinovich M E, Ionescu A. Windows Internals, Part 1[M]. Microsoft Press, 2012.
- [7] Tanenbaum A S, Bos H. Modern Operating Systems[M]. 4th ed. Pearson, 2014.
- [8] Shannon C E. A Mathematical Theory of Communication[J]. The Bell System Technical Journal, 1948, 27(3): 379-423.
- [9] Aho A V, Lam M S, Sethi R, et al. Compilers: Principles, Techniques, and Tools[M]. 2nd ed. Pearson Education, 2006.
- [10] Google Cloud. Context Caching Overview[EB/OL]. Vertex AI Documentation, 2024.
- [11] DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model[J]. arXiv preprint arXiv:2405.04434, 2024.
- [12] Anthropic. Introducing Tool Use and Caching[EB/OL]. Anthropic Blog, 2024.
- [13] Regulation (EU) 2016/679 of the European Parliament and of the Council on the Protection of Natural Persons with Regard to the Processing of Personal Data and on the Free Movement of Such Data (General Data Protection Regulation)[S]. 2016.

[14] Saltzer J H, Schroeder M D. The Protection of Information in Computer Systems[J]. Proceedings of the IEEE, 1975, 63(9): 1278-1308.

[15] Minsky M. A Framework for Representing Knowledge[R]. MIT-AI Laboratory Memo 306, 1974.

[16] Cousot P, Cousot R. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints[C]//Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages. 1977: 238-252.

[17] Engelbart D C. Augmenting Human Intellect: A Conceptual Framework[R]. SRI Summary Report AFOSR-3223, 1962.

[18] Denis M, Lleras M. A Survey on the Memorization of Large Language Models[J]. arXiv preprint arXiv:2308.05544, 2023.

[19] Wei J, Wang X, Schuurmans D, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models[J]. Advances in Neural Information Processing Systems, 2022, 35: 24824-24837.

[20] Bubeck S, Chandrasekaran V, Eldan R, et al. Sparks of Artificial General Intelligence: Early Experiments with GPT-4[J]. arXiv preprint arXiv:2303.12712, 2023.

[21] Liu X, Yu H, Zhang J, et al. AgentBench: Evaluating Large Language Models as Agents[C]//The Twelfth International Conference on Learning Representations. 2024.

[22] Wang G, Xie Y, Jiang Y, et al. Voyager: An Open-Ended Embodied Agent with Large Language Models[J]. arXiv preprint arXiv:2305.16291, 2023.

[23] FIPA. FIPA ACL Message Structure Specification[S]. Foundation for Intelligent Physical Agents, 2002.

[24] Xu Y, Zhuo H, Chen Q, et al. GraphRAG: Structured Retrieval-Augmented Generation for Knowledge-Intensive LLM Applications[J]. arXiv preprint arXiv:2404.16130, 2024.

附录

附录 A 标准化工单协议完整 Schema

见 5.2 节完整定义。

附录 B 注册表标准根键规范

根键名称	前缀	核心用途	修改权限
HKEY_LOCAL_AGENT	HKLA	Agent 自身核心配置	内核态，仅初始化时修改
HKEY_CURRENT_USER	HKCU	当前用户信息与	内核态可写，用

		偏好	户态可读
HKEY_LOCAL_MACHINE	HKLM	宿主环境配置	系统管理员
HKEY_CURRENT_SESSION	HKCS	当前会话运行时状态	内核态可写
HKEY_CLOUD_SYNC	HKCSY	云端同步状态	同步服务

附录 C 系统调用接口完整定义

见 4.3 节完整定义。

附录 D 术语表

术语	英文	定义
AgentOS	Agent Operating System	智能体操作系统，本文提出的分层架构体系
注册表	Registry	树形结构化键值库，存储确定性元数据与状态指针
编译器	Compiler	意图净化与指令翻译组件，连接自然语言与结构化协议
工单	Work Ticket	结构化的机间通信单元，是 Agent 与模型 / 其他 Agent 的交互载体
云内存	Cloud Memory	与推理引擎同址部署的专用内存空间，驻留用户认知资产
四表分离	Four-Table Separation	注册表、环境变量、系统日志、记忆库逻辑分离的记忆模型

三大分离原则	Three Separation Principles	配置与材料分离、指针与数据分离、陪伴与执行分离
缓存握手协议	Cache Handshake Protocol	管理云端缓存资产生命周期的标准化协议